

Laporan Tugas Kecil 3

IF2211 STRATEGI ALGORITMA

PENYELESAIAN ICE SLIDING PUZZLE DENGAN UCS, GBFS, DAN A*

SEMESTER II TAHUN 2025/2026



NARENDRA DHARMA WISTARA M.
13524044

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1 Deskripsi Penugasan

Ice Sliding Puzzle adalah permainan pencarian lintasan pada papan dua dimensi. Pemain berada pada suatu petak awal dan harus mencapai petak tujuan. Setiap kali pemain memilih arah gerak, pemain akan terus meluncur pada arah tersebut sampai bertemu penghalang. Selama meluncur, pemain dapat melewati beberapa jenis petak, seperti petak biasa, checkpoint, lava, dan dinding. Jika pemain keluar papan atau melewati lava, langkah tersebut dianggap tidak valid.

Tugas kecil ini meminta dibuatnya program yang dapat menyelesaikan Ice Sliding Puzzle dengan pendekatan pencarian graf. Program menerima konfigurasi papan dari berkas teks, membangun ruang status permainan secara implisit, lalu mencari urutan gerakan yang membawa pemain dari posisi awal menuju tujuan. Jika terdapat checkpoint, checkpoint harus dilewati sesuai urutan sebelum permainan dianggap selesai.

2 Spesifikasi Tugas

2.1 Spesifikasi Wajib

Program wajib memenuhi kebutuhan berikut.

- Membaca masukan papan dari berkas `.txt`.
- Memvalidasi format papan, posisi awal, posisi tujuan, checkpoint, dan matriks biaya.
- Menyelesaikan persoalan menggunakan algoritma wajib UCS, GBFS, dan A*.
- Menampilkan solusi berupa urutan gerakan, total biaya, jumlah iterasi, waktu eksekusi, dan visualisasi langkah solusi.
- Menyediakan penyimpanan hasil solusi ke berkas `.txt`.
- Menyediakan antarmuka CLI untuk menjalankan solver dari terminal.

Format umum berkas masukan adalah sebagai berikut.

```
N M
<N baris grid>
<N x M nilai cost>
```

Karakter pada grid memiliki arti sebagai berikut.

Karakter	Keterangan
X	Dinding. Pemain berhenti sebelum memasuki petak ini.
*	Petak biasa atau ice tile.
L	Lava. Langkah yang melewati petak ini tidak valid.
Z	Posisi awal pemain.
0	Posisi tujuan.
0-9	Checkpoint yang harus dilewati berurutan.

Tabel 1: Daftar karakter pada papan

2.2 Spesifikasi Bonus

Pada implementasi ini juga dibuat beberapa fitur bonus. Pertama, GUI web dibuat agar pengguna dapat mengunggah testcase, memilih algoritma, memilih heuristic untuk GBFS dan A*, melihat animasi solusi per langkah, serta melihat trace graf pencarian yang terbentuk selama solver berjalan. GUI juga menyediakan tombol untuk menyimpan hasil solusi ke berkas `.txt`. Kedua, program menambahkan algoritma pathfinding alternatif BFS dan DFS. Ketiga, selain heuristic utama, program menyediakan beberapa alternatif heuristic untuk GBFS dan A*.

3 Dasar Teori

3.1 Representasi Graf

Persoalan Ice Sliding Puzzle dapat dipandang sebagai persoalan pencarian lintasan pada graf berbobot. Setiap simpul menyatakan suatu status permainan, yaitu posisi pemain dan checkpoint berikutnya yang harus dicapai. Setiap sisi menyatakan satu aksi slide ke salah satu arah. Bobot sisi adalah total biaya petak yang dilewati selama slide tersebut.

Graf tidak perlu dibangun seluruhnya sejak awal. Program cukup membangkitkan tetangga dari simpul yang sedang diekspansi. Pendekatan ini disebut pembangkitan graf implisit dan cocok untuk ruang status yang dapat membesar secara cepat.

3.2 Uniform Cost Search

Uniform Cost Search (UCS) adalah algoritma pencarian yang selalu mengekspansi simpul dengan biaya lintasan terkecil dari simpul awal. Jika semua biaya sisi tidak negatif, UCS menjamin solusi optimal berdasarkan total biaya. Prioritas simpul pada UCS didefinisikan sebagai berikut.

$$f(n) = g(n)$$

dengan $g(n)$ adalah biaya lintasan dari simpul awal menuju simpul n .

3.3 Greedy Best First Search

Greedy Best First Search (GBFS) memilih simpul berdasarkan nilai heuristic yang memperkirakan jarak menuju target. Algoritma ini cenderung cepat menuju daerah yang dianggap menjanjikan, tetapi tidak menjamin solusi optimal karena biaya lintasan yang sudah ditempuh tidak menjadi bagian dari prioritas utama. Prioritas GBFS adalah:

$$f(n) = h(n)$$

3.4 A*

A* menggabungkan biaya aktual yang sudah ditempuh dengan estimasi biaya menuju target. Prioritas simpul pada A* adalah:

$$f(n) = g(n) + h(n)$$

Jika heuristic yang digunakan admissible dan konsisten, A* dapat menemukan solusi optimal dengan jumlah ekspansi yang umumnya lebih sedikit dibanding UCS.

3.5 Breadth First Search

Breadth First Search (BFS) adalah algoritma pencarian tidak berbobot yang mengekspansi simpul berdasarkan kedalaman. Pada implementasi ini, setiap aksi slide dianggap sebagai satu langkah, sehingga BFS optimal terhadap jumlah slide, bukan terhadap total cost traversal.

$$f(n) = \text{depth}(n)$$

3.6 Depth First Search

Depth First Search (DFS) menggunakan stack untuk menelusuri cabang sedalam mungkin sebelum mencoba cabang lain. DFS dapat menemukan solusi pada ruang status finite, tetapi tidak menjamin solusi dengan jumlah slide paling sedikit maupun cost paling kecil.

3.7 Heuristic

Program menyediakan tiga mode heuristic.

1. H1: heuristic utama berupa jarak Manhattan dari posisi sekarang ke target berikutnya.
2. H2: alternatif heuristic berupa jarak Manhattan ke target berikutnya dikalikan biaya petak minimum yang dapat dilewati.
3. H3: alternatif heuristic berupa total jarak Manhattan menuju checkpoint yang tersisa secara berurutan, lalu menuju goal.

Pada permainan slide, jarak Manhattan bukan jarak langkah sebenarnya karena satu gerakan dapat melewati beberapa petak. Namun, heuristic ini tetap berguna sebagai estimasi arah pencarian yang sederhana dan murah dihitung.

Untuk A*, admissibility heuristic pada permainan ini tidak selalu dapat dijamin secara umum. Satu aksi slide dapat melewati beberapa petak sekaligus, sehingga jarak Manhattan dalam satuan petak dapat lebih besar daripada biaya aksi minimum jika terdapat tile berbiaya rendah atau jika satu slide mendekatkan posisi melewati banyak petak. H1 dan H2 tetap dipakai karena murah dihitung dan memberi arah pencarian yang baik, tetapi optimalitas A* secara teoritis hanya terjamin pada testcase ketika nilai heuristic tidak melebihi biaya sisa sebenarnya. H3 lebih informatif untuk checkpoint berurutan karena menjumlahkan estimasi ke target-target yang tersisa, namun juga tidak dijamin admissible pada seluruh konfigurasi papan karena tidak memodelkan titik berhenti akibat dinding dan efek slide.

4 Desain

4.1 Tech Stack

Aplikasi dirancang menggunakan Go untuk bagian parser, model puzzle, dan solver. Untuk bonus GUI, aplikasi menggunakan backend HTTP Go dan frontend React berbasis Vite.

1. Bahasa pemrograman solver: Go 1.26.2.
2. Backend GUI: Go HTTP server.
3. Frontend GUI: React dan Vite.
4. Kontainerisasi: Docker multi-stage build.

4.2 Struktur Program

Struktur utama repository adalah sebagai berikut.

1. `src/main.go`: entry point CLI.
2. `src/parser/`: parser dan validasi berkas input.
3. `src/game/`: representasi state permainan, aturan slide, dan pembangkitan successor.
4. `src/solver/`: implementasi UCS, GBFS, A*, BFS, DFS, priority queue, dan heuristic.
5. `src/searchtrace/`: struktur data trace pencarian untuk kebutuhan visualisasi GUI.
6. `src/web/backend/`: backend HTTP untuk menerima file input dan menjalankan solver.
7. `src/web/frontend/`: antarmuka React untuk upload testcase, kontrol solver, playback solusi, dan graf pencarian.
8. `test/`: tempat penyimpanan testcase.
9. `doc/`: spesifikasi dan laporan.

4.3 Representasi Data

Konfigurasi papan direpresentasikan oleh struktur `BoardConfig`. Struktur ini menyimpan ukuran papan, grid karakter, matriks biaya, posisi awal, posisi tujuan, dan posisi checkpoint.

```
1 type BoardConfig struct {
2     N, M          int
3     Grid          [][]rune
4     Cost          [][]int
5     StartRow      int
6     StartCol      int
7     GoalRow       int
8     GoalCol       int
9     NumCheckpoints int
10    CheckpointPos map[int][2]int
11 }
```

Status pencarian direpresentasikan oleh `game.Node`. Selain posisi pemain, state juga menyimpan checkpoint berikutnya yang harus dicapai. Dengan demikian, posisi yang sama dapat menjadi state yang berbeda jika progres checkpoint-nya berbeda.

```
1 type Node struct {
2     Row, Col    int
3     NextTarget int
4 }
```

4.4 Model Gerakan

Gerakan pada puzzle tidak berpindah satu petak, melainkan slide sampai petak berikutnya adalah dinding. Fungsi slide memeriksa arah gerak, menambahkan biaya petak yang dilewati, memperbarui checkpoint, dan menolak langkah yang keluar papan, melewati lava, atau melewati checkpoint yang belum boleh dilewati.

Setiap successor yang valid direpresentasikan sebagai `Edge`.

```
1 type Edge struct {
2     State      Node
3     Cost       int
4     Direction  Dir
5     Path       []Pos
6 }
```

4.5 Desain Solver

Solver menggunakan satu kerangka pencarian umum, yaitu `graphSearch`. Perbedaan UCS, GBFS, dan A* hanya terletak pada cara menghitung prioritas simpul.

Algoritma	Prioritas
UCS	$g(n)$
GBFS	$h(n)$
A*	$g(n) + h(n)$

Tabel 2: Fungsi prioritas pada solver

Frontier disimpan dalam priority queue. Program juga menyimpan `bestCost` untuk setiap state agar state yang pernah dicapai dengan biaya lebih murah tidak diproses ulang menggunakan biaya yang lebih mahal.

4.6 Desain BFS dan DFS

BFS dan DFS diimplementasikan dengan kerangka pencarian tidak berbobot. Keduanya tetap memakai fungsi `GetSuccessors` yang sama dengan UCS, GBFS, dan A*. Perbedaannya terletak pada struktur frontier.

Algoritma	Frontier	Karakteristik
BFS	Queue	Menemukan solusi dengan jumlah slide minimum, tetapi tidak menjamin total cost minimum.
DFS	Stack	Menelusuri cabang sedalam mungkin dan tidak menjamin optimalitas.

Tabel 3: Desain frontier BFS dan DFS

Pada BFS dan DFS, state yang sudah pernah dikunjungi tidak dimasukkan kembali ke frontier untuk mencegah siklus. Total cost tetap dihitung sepanjang lintasan yang ditemukan agar hasilnya dapat dibandingkan dengan algoritma berbobot.

4.7 Cara Kompilasi dan Menjalankan Program

Program membutuhkan Go untuk menjalankan CLI dan backend. Frontend GUI membutuhkan Node.js dan npm jika dijalankan tanpa Docker. Semua perintah berikut dijalankan dari root repository.

Untuk melakukan kompilasi executable:

```
1 go build -o bin/ice-sliding-solver ./src
```

Untuk menjalankan hasil kompilasi:

```
1 ./bin/ice-sliding-solver
```

Untuk menjalankan CLI:

```
1 go run ./src
```

Setelah solver selesai, CLI menawarkan penyimpanan hasil ke berkas `.txt`. Berkas hasil berisi algoritma, heuristic jika ada, status solusi, urutan gerakan, total cost, jumlah iterasi, waktu eksekusi, dan visualisasi papan per langkah.

Untuk menjalankan backend GUI tanpa Docker:

```
1 go run ./src/web/backend
```

Untuk menjalankan frontend GUI tanpa Docker:

```
1 cd src/web/frontend
2 npm install
3 npm run dev
```

Untuk menjalankan GUI menggunakan Docker:

```
1 docker compose down
2 docker compose up --build
```

GUI dapat diakses melalui alamat berikut.

```
1 http://localhost:8080
```

5 Implementasi Spesifikasi Wajib

5.1 Parser dan Validasi Input

Parser berada pada package `parser`. Fungsi utama parser adalah `ParseFile`. Fungsi ini menerima path berkas `.txt`, membaca seluruh isi file, lalu membentuk `BoardConfig`. Validasi yang dilakukan meliputi:

- ekstensi file harus `.txt`;
- baris pertama harus berisi tepat dua bilangan bulat positif N M ;
- jumlah baris grid harus sesuai dengan N ;
- setiap baris grid harus memiliki panjang M ;
- papan harus memiliki tepat satu start Z dan satu goal O ;
- checkpoint tidak boleh duplikat dan harus berurutan mulai dari 0 ;
- jumlah nilai cost harus tepat $N * M$;
- setiap nilai cost harus berupa bilangan bulat tidak negatif.

Dengan validasi tersebut, solver dapat berasumsi bahwa struktur papan yang diterima sudah konsisten.

5.2 Pembangkitan Successor

Package `game` berisi aturan permainan. Fungsi `GetSuccessors` membangkitkan semua langkah valid dari suatu state dengan mencoba empat arah: atas, bawah, kiri, dan kanan.

```
1 func GetSuccessors(board *parser.BoardConfig, s Node) []Edge {
2     var successors []Edge
3     for _, dir := range AllDirs {
4         newState, cost, path, valid := Slide(board, s, dir)
5         if valid {
6             successors = append(successors, Edge{
7                 State:    newState,
8                 Cost:      cost,
9                 Direction: dir,
10                Path:      path,
11            })
12        }
13    }
14    return successors
15 }
```

Fungsi `Slide` merupakan inti aturan permainan. Selama pemain belum bertemu dinding, fungsi ini terus memindahkan posisi ke petak berikutnya, menjumlahkan biaya, dan mencatat path petak yang dilewati. Jika gerakan keluar papan atau melewati lava, gerakan langsung ditolak. Jika bertemu checkpoint, checkpoint hanya diterima apabila checkpoint tersebut merupakan target berikutnya atau checkpoint yang sudah dilewati sebelumnya.

5.3 Kerangka Pencarian

Solver menggunakan struktur `SearchNode` untuk menyimpan state permainan dan informasi pencarian.

```
1 type SearchNode struct {
2     Node      graph.Node
3     Parent    *SearchNode
4     Move      graph.Dir
5     GCost     int
6     HCost     int
7     Priority   int
8     StepPath  []graph.Pos
9     Index     int
10 }
```

Field `Parent`, `Move`, dan `StepPath` digunakan untuk merekonstruksi solusi setelah goal ditemukan. Field `GCost`, `HCost`, dan `Priority` digunakan oleh priority queue.

Secara garis besar, algoritma pencarian berjalan sebagai berikut.

1. Buat state awal dari posisi Z dengan `NextTarget = 0`.
2. Masukkan state awal ke priority queue.
3. Selama priority queue tidak kosong, ambil simpul dengan prioritas terkecil.
4. Abaikan simpul jika sudah ada rute yang lebih murah menuju state yang sama.
5. Jika simpul adalah goal, rekonstruksi path dan kembalikan hasil.
6. Jika belum goal, bangkitkan semua successor valid.
7. Untuk setiap successor, hitung biaya baru, heuristic, dan prioritas sesuai algoritma yang dipilih.
8. Masukkan successor ke priority queue apabila rute tersebut lebih baik daripada rute sebelumnya.

5.4 Implementasi UCS, GBFS, dan A*

Tiga algoritma wajib diimplementasikan sebagai wrapper terhadap `graphSearch`. Wrapper menjaga antarmuka pemanggilan tetap sederhana.


```

1 func UCS(board *parser.BoardConfig) Result {
2     return graphSearch(board, AlgorithmUCS, "")
3 }
4
5 func GBFS(board *parser.BoardConfig, heuristicMode string) Result {
6     return graphSearch(board, AlgorithmGBFS, heuristicMode)
7 }
8
9 func AStar(board *parser.BoardConfig, heuristicMode string) Result {
10    return graphSearch(board, AlgorithmAStar, heuristicMode)
11 }

```

Perhitungan prioritas dilakukan pada satu fungsi agar perilaku ketiga algoritma mudah dibandingkan.

```

1 func calculatePriority(algorithm Algorithm, gCost, hCost int) int {
2     switch algorithm {
3     case AlgorithmUCS:
4         return gCost
5     case AlgorithmGBFS:
6         return hCost
7     case AlgorithmAStar:
8         return gCost + hCost
9     default:
10        return gCost
11    }
12 }

```

5.5 Implementasi BFS dan DFS

BFS dan DFS diimplementasikan sebagai algoritma bonus pada fungsi `unweightedGraphSearch`. Fungsi ini menggunakan representasi state dan successor yang sama dengan algoritma wajib, tetapi tidak menggunakan priority queue. BFS mengambil elemen dari depan slice seperti queue, sedangkan DFS mengambil elemen terakhir seperti stack.

```

1 func BFS(board *parser.BoardConfig) Result {
2     return unweightedGraphSearch(board, AlgorithmBFS, nil)
3 }
4
5 func DFS(board *parser.BoardConfig) Result {
6     return unweightedGraphSearch(board, AlgorithmDFS, nil)
7 }

```

Pada BFS, kedalaman simpul menyatakan jumlah slide yang telah dilakukan. Oleh karena itu, BFS optimal terhadap jumlah slide. Namun, karena cost setiap slide bergantung pada total cost tile yang dilewati, BFS tidak selalu menghasilkan total cost terkecil. DFS digunakan sebagai pembanding karena dapat menemukan solusi dengan strategi penelusuran yang berbeda, tetapi tidak menjamin optimalitas.

5.6 Heuristic

Heuristic digunakan oleh GBFS dan A*. Program menyediakan tiga pilihan.

1. H1: Manhattan normal ke target berikutnya.
2. H2: alternatif heuristic Manhattan ke target berikutnya dikali biaya minimum tile yang dapat dilewati.
3. H3: alternatif heuristic total Manhattan dari posisi saat ini menuju semua checkpoint tersisa secara berurutan, lalu menuju goal.

```

1 func Heuristic(board *parser.BoardConfig, node graph.Node, mode string) int {
2     switch mode {
3     case HeuristicManhattan:
4         return manhattanToNextTarget(board, node)
5     case HeuristicManhattanCost:
6         return manhattanToNextTarget(board, node) * minTraversableCost(board)
7     case HeuristicRemaining:

```

```

8     return remainingManhattan(board, node)
9     default:
10        return manhattanToNextTarget(board, node) * minTraversableCost(board)
11    }
12 }

```

5.7 Rekonstruksi Solusi

Ketika goal ditemukan, solver tidak langsung memiliki urutan gerakan dari awal ke akhir. Setiap `SearchNode` hanya menyimpan parent. Oleh karena itu, program melakukan traversal mundur dari goal ke start, menyimpan simpul dan gerakan yang dilewati, lalu membalik urutan hasil.

Hasil akhir disimpan dalam struktur `Result`.

```

1 type Result struct {
2     Found          bool
3     Moves           []graph.Dir
4     PathNodes       []graph.Node
5     StepPaths       [][]graph.Pos
6     TotalCost       int
7     Iterations      int
8     ExecutionMs     int64
9 }

```

5.8 CLI

CLI berada pada `src/main.go`. Program meminta pengguna memasukkan path file testcase, pilihan algoritma, dan pilihan heuristic jika algoritma yang dipilih adalah GBFS atau A*. Setelah solver selesai, CLI menampilkan status solusi, urutan gerakan, total cost, jumlah iterasi, waktu eksekusi, dan visualisasi papan untuk setiap langkah solusi.

5.9 Penyimpanan Solusi

Setelah hasil pencarian ditampilkan, CLI menanyakan apakah pengguna ingin menyimpan hasil ke berkas `.txt`. Jika pengguna memilih ya, program meminta path output. Ekstensi `.txt` ditambahkan secara otomatis apabila pengguna belum menuliskannya.

Berkas solusi yang disimpan berisi ringkasan algoritma, heuristic jika digunakan, status solusi, urutan gerakan, total cost, jumlah iterasi, waktu eksekusi, dan visualisasi papan untuk setiap langkah. Jika solusi tidak ditemukan, program tetap dapat menyimpan status *not found*, jumlah iterasi, waktu eksekusi, dan papan awal.

6 Implementasi Spesifikasi Bonus

6.1 Algoritma Pathfinding Alternatif

Bonus algoritma pathfinding alternatif dipenuhi dengan menambahkan BFS dan DFS. Keduanya dapat dijalankan dari CLI maupun GUI. Pada GUI, frontend mengirim nilai BFS atau DFS melalui field `algorithm`, lalu backend memanggil `BFSWithTrace` atau `DFSWithTrace`.

```

1 case "BFS":
2     result, trace := solver.BFSWithTrace(board)
3     return result, trace, nil
4 case "DFS":
5     result, trace := solver.DFSWithTrace(board)
6     return result, trace, nil

```

Kedua algoritma ini tidak memerlukan heuristic, sehingga dropdown heuristic pada GUI hanya aktif untuk GBFS dan A*.

6.2 GUI Web

GUI dibuat sebagai aplikasi web agar solver dapat digunakan tanpa interaksi terminal. Arsitektur GUI terdiri dari dua bagian.

1. Backend Go pada `src/web/backend/server.go`.
2. Frontend React pada `src/web/frontend`.

Backend menyediakan endpoint `POST /solve`. Endpoint ini menerima `multipart/form-data` berisi file testcase, nama algoritma, dan mode heuristic. Setelah request diterima, backend menjalankan parser dan solver, lalu mengembalikan respons JSON.

```
1 POST /solve
2 file=<testcase.txt>
3 algorithm=UCS|GBFS|ASTAR|BFS|DFS
4 heuristic=H1|H2|H3
```

Respons backend berisi informasi berikut.

- `algorithm`: algoritma yang dijalankan.
- `heuristic`: heuristic yang dipakai untuk GBFS atau A*.
- `found`: status apakah solusi ditemukan.
- `moves`: urutan gerakan solusi.
- `totalCost`: total biaya solusi.
- `iterations`: jumlah simpul yang diekspansi.
- `executionMs`: waktu eksekusi solver dalam milidetik.
- `steps`: papan visual untuk setiap langkah solusi.
- `trace`: node dan edge untuk visualisasi graf pencarian.

6.3 Frontend React

Frontend menyediakan beberapa komponen utama.

1. `ControlPanel.jsx`: form upload file, pilihan algoritma, dan pilihan heuristic.
2. `Board.jsx`: visualisasi grid puzzle.
3. `ResultPanel.jsx`: ringkasan hasil solver.
4. `PlaybackControls.jsx`: kontrol previous, next, play/pause, slider step, dan speed.
5. `GraphView.jsx`: visualisasi trace pencarian dalam bentuk graf.

Alur penggunaan GUI adalah sebagai berikut.

1. Pengguna mengunggah file testcase `.txt`.
2. Pengguna memilih algoritma UCS, GBFS, A*, BFS, atau DFS.
3. Jika memilih GBFS atau A*, pengguna memilih heuristic.
4. Frontend mengirim request ke endpoint `/solve`.
5. Backend mengembalikan hasil solver.
6. Frontend menampilkan hasil, playback solusi, graf pencarian, dan tombol `Save .txt` untuk menyimpan ringkasan solusi.

6.4 Trace Graf Pencarian

Untuk kebutuhan GUI, solver memiliki varian fungsi `WithTrace`. Trace tidak dimasukkan ke dalam `solver.Result` agar struktur hasil utama tetap fokus pada solusi pencarian. Data trace diletakkan pada package terpisah, yaitu `searchtrace`.

```
1 type Step struct {  
2     Expanded Node  
3     Edges     []Edge  
4 }
```

Setiap `Step` merepresentasikan satu ekspansi simpul. Edge pada trace menyimpan informasi arah, biaya langkah, nilai `g`, nilai `h`, prioritas, dan apakah edge tersebut diterima masuk frontier. Dengan informasi ini, GUI dapat membedakan successor yang dipakai solver dan successor yang ditolak karena sudah ada rute lebih murah.

6.5 Alternatif Heuristic

Bonus alternatif heuristic dipenuhi dengan menyediakan H2 dan H3 selain heuristic utama H1. Ketiga heuristic dapat dipilih pada GUI ketika algoritma yang digunakan adalah GBFS atau A*.

6.6 Docker

Dockerfile menggunakan multi-stage build.

1. Stage Node membangun frontend React.
2. Stage Go membangun binary backend.
3. Stage Alpine menjalankan backend dan menyajikan hasil build frontend sebagai static SPA.

Konfigurasi `docker-compose.yml` di root repository mengatur build image dari `src/Dockerfile` dan memetakan port 8080. Dengan desain ini, pengguna cukup menjalankan `docker compose down` lalu `docker compose up --build` untuk mengakses GUI lengkap melalui browser.

7 Pengujian

Pengujian pada laporan ini dilakukan melalui GUI. Setiap bukti yang dimasukkan ke laporan berupa tangkapan layar GUI yang memperlihatkan file testcase yang diunggah, algoritma yang dipilih, heuristic jika digunakan, panel hasil, board solusi, playback, dan search graph. Untuk membuktikan fitur penyimpanan solusi, salah satu bukti GUI juga dapat memperlihatkan tombol `Save .txt` atau hasil unduhan berkas solusi. Testcase utama menggunakan nama file yang sudah disiapkan pada folder `test/` sesuai algoritma yang diuji. File `testcase_ucs.txt` dan `testcase_astar.txt` memiliki konfigurasi papan yang sama, tetapi disimpan sebagai file berbeda agar bukti pengujian per algoritma lebih jelas. Demikian pula `testcase_dfs.txt` dan `testcase_bfs.txt`. `testcase_sanity.txt` tetap tersedia sebagai sanity check sederhana untuk upload dan rendering GUI, tetapi tidak dijadikan bukti utama karena kasusnya terlalu ringan dibandingkan testcase lain.

7.1 Rencana Pengujian GUI

No	File	Algoritma	Tujuan
1	testcase_ucs.txt	UCS	Menguji pencarian berbasis cost pada papan dengan checkpoint dan lava.
2	testcase_gbfs.txt	GBFS H1, H2, H3	Menguji seluruh heuristic GBFS pada papan lebih besar dengan tiga checkpoint.
3	testcase_astar.txt	A* H1, H2, H3	Menguji seluruh heuristic A* pada papan berbobot dengan checkpoint.
4	testcase_dfs.txt	DFS	Menguji algoritma bonus DFS pada papan non-persegi.
5	testcase_bfs.txt	BFS	Menguji algoritma bonus BFS pada papan non-persegi.

Tabel 4: Rencana pengujian algoritma melalui GUI

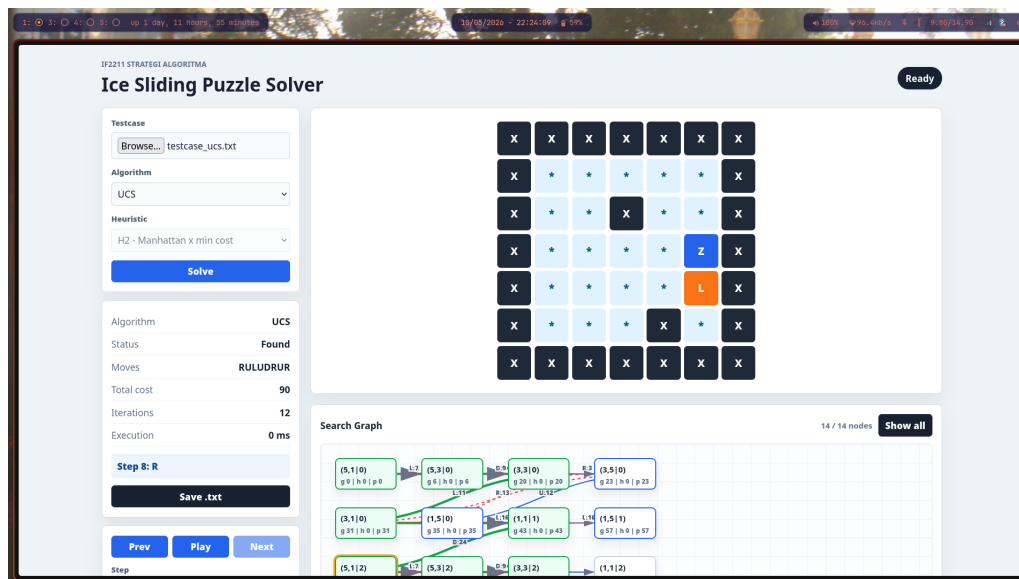
7.2 Format Bukti Pengujian

Setiap bukti GUI dilengkapi dengan informasi berikut.

- file testcase yang diunggah;
- algoritma dan heuristic yang dipilih;
- status solusi ditemukan;
- moves, total cost, jumlah iterasi, dan waktu eksekusi;
- tampilan board pada salah satu step playback;
- tampilan search graph;
- tombol atau hasil penyimpanan solusi .txt untuk minimal satu pengujian.

7.3 Kasus Uji 1: UCS pada testcase_ucs.txt

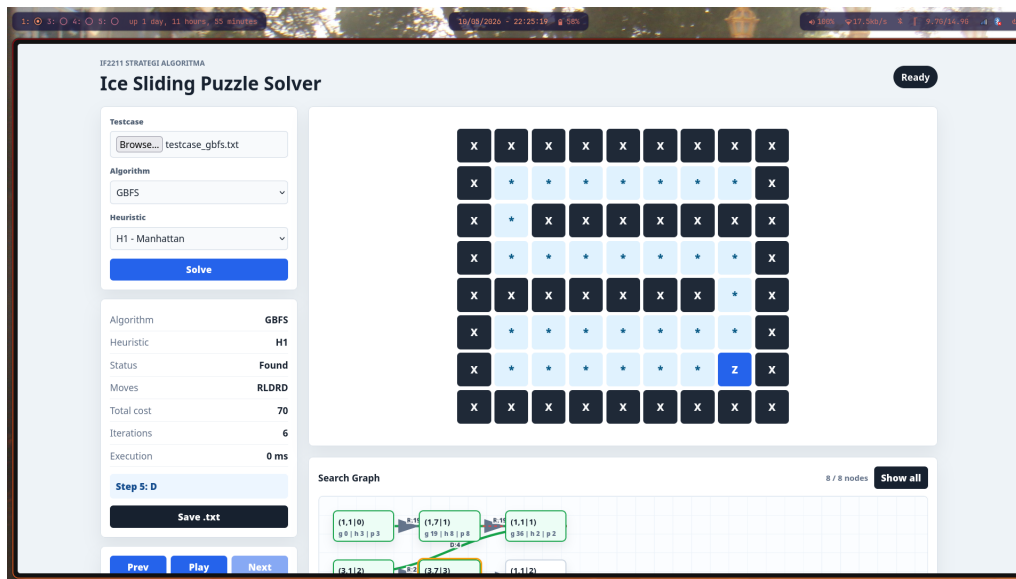
- File testcase: test/testcase_ucs.txt.
- Algoritma: UCS.
- Heuristic: tidak digunakan.
- Kompleksitas kasus: papan 7×7 , terdapat checkpoint 0 dan 1, lava, dinding, serta cost tile yang bervariasi.
- Ekspektasi: GUI menampilkan solusi dengan total cost minimum berdasarkan UCS.



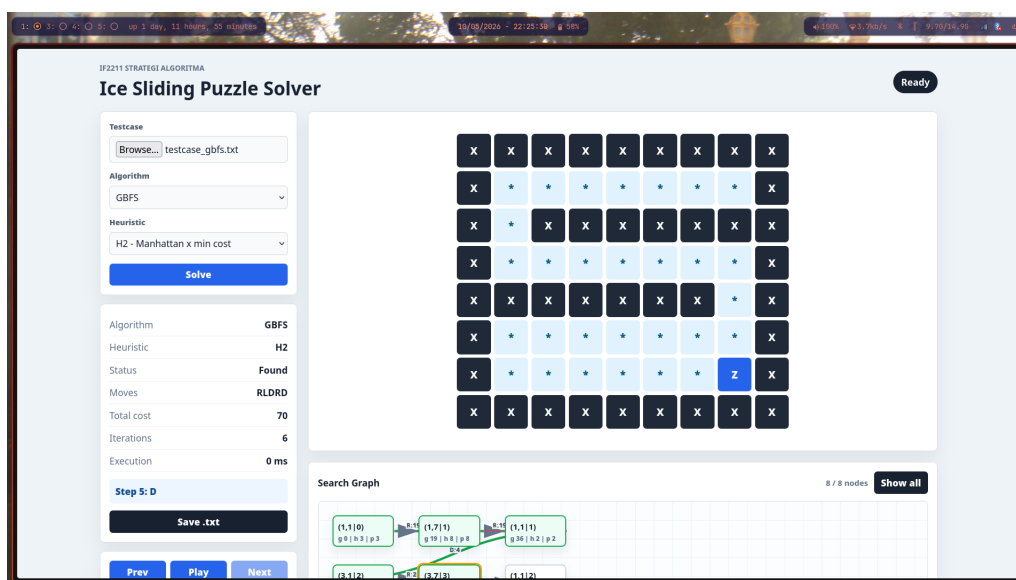
Gambar 1: Bukti GUI UCS pada testcase_ucs.txt

7.4 Kasus Uji 2: GBFS H1, H2, dan H3 pada testcase_gbfs.txt

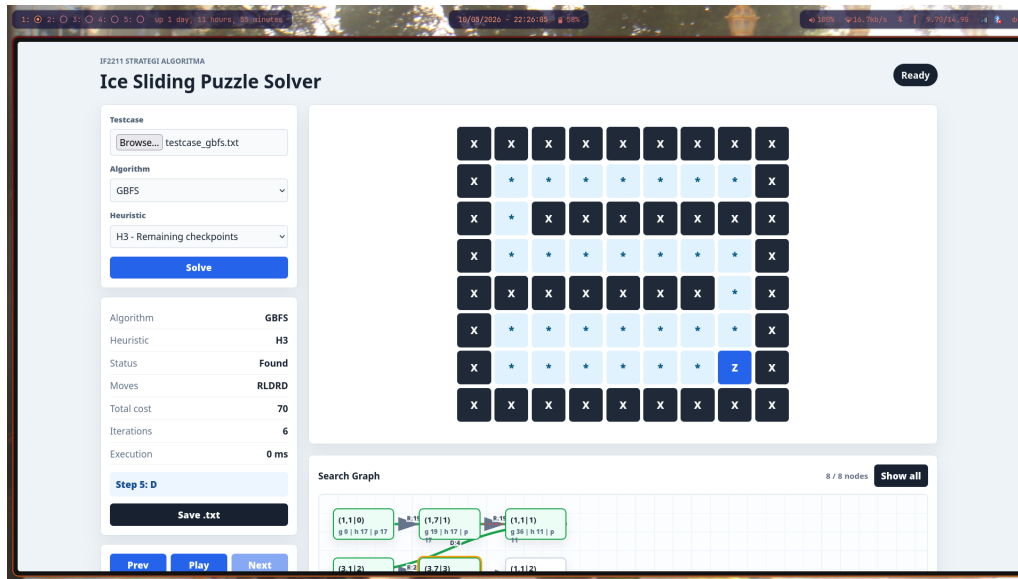
- File testcase: test/testcase_gbfs.txt.
- Algoritma: GBFS.
- Heuristic yang diuji: H1, H2, dan H3.
- Kompleksitas kasus: papan 8×9 dengan checkpoint 0, 1, dan 2; jalur dibatasi oleh banyak dinding sehingga urutan checkpoint harus benar.
- Ekspektasi: setiap heuristic dapat menghasilkan respons valid dan GUI menampilkan solusi beserta search graph.



Gambar 2: Bukti GUI GBFS H1 pada testcase_gbfs.txt



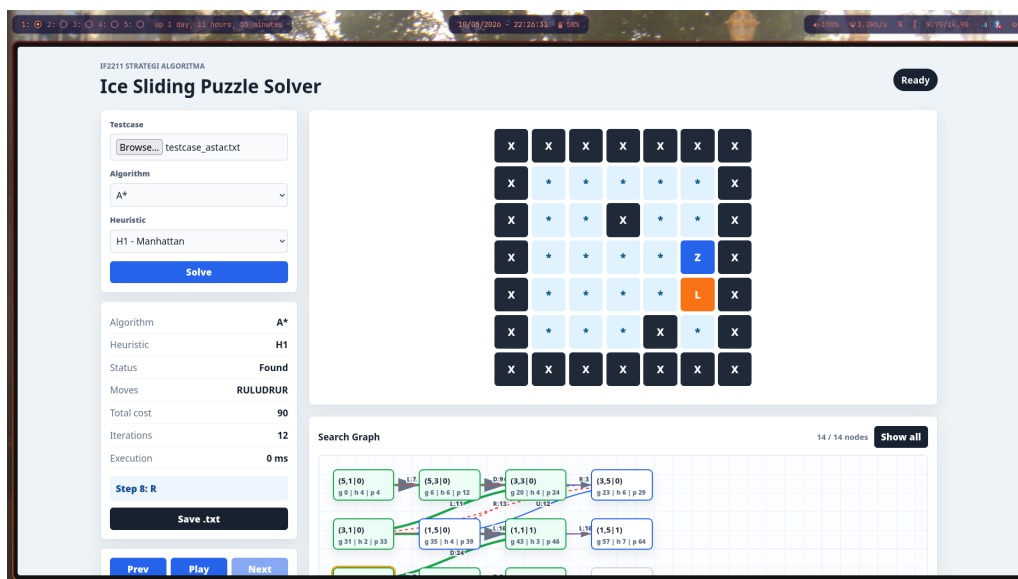
Gambar 3: Bukti GUI GBFS H2 pada testcase_gbfs.txt



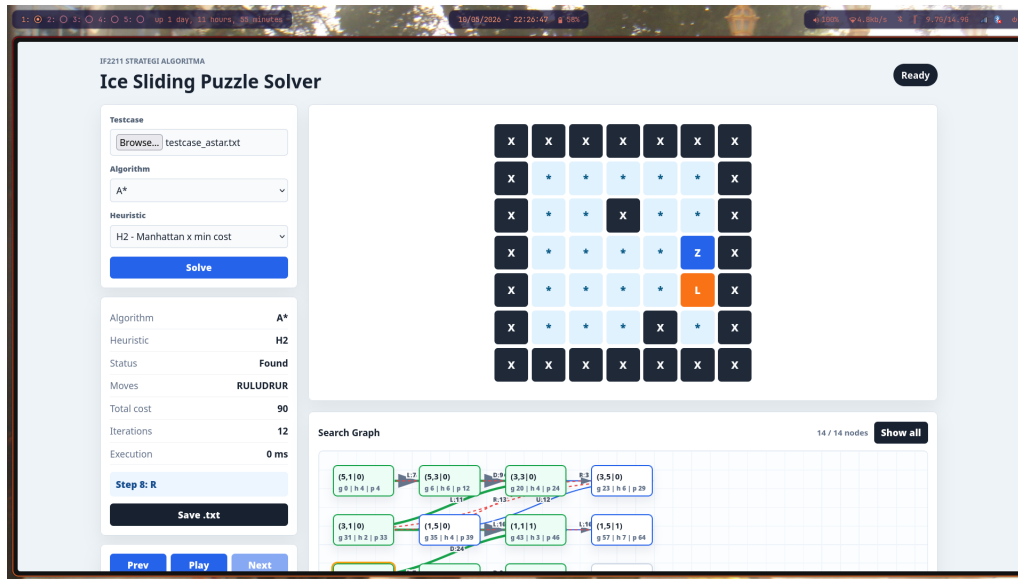
Gambar 4: Bukti GUI GBFS H3 pada testcase_gbfs.txt

7.5 Kasus Uji 3: A* H1, H2, dan H3 pada testcase_astar.txt

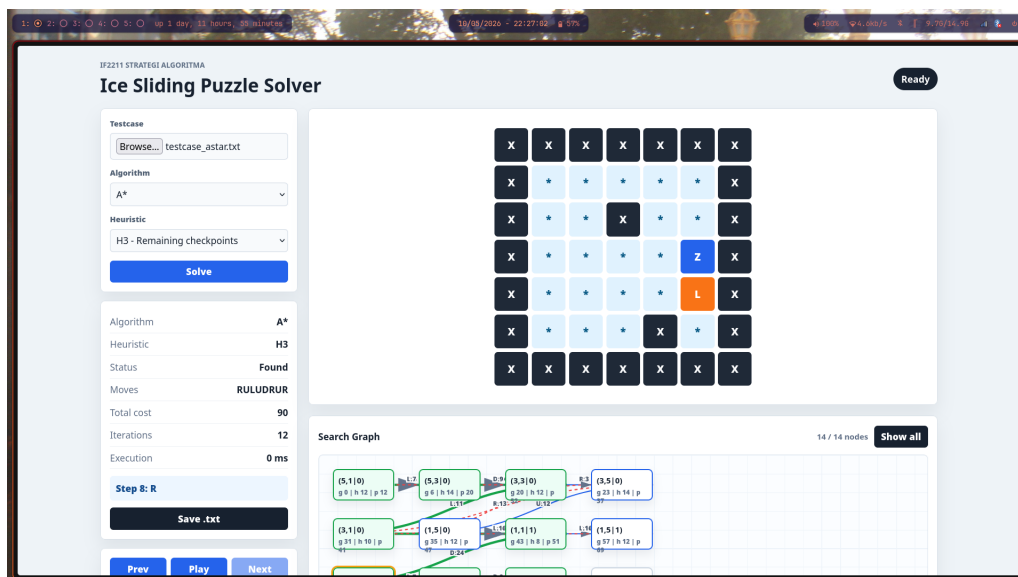
- File testcase: test/testcase_astar.txt.
- Algoritma: A*.
- Heuristic yang diuji: H1, H2, dan H3.
- Kompleksitas kasus: papan berbobot dengan checkpoint dan lava, sehingga A* harus memperhatikan $g(n)$ dan $h(n)$.
- Ekspektasi: setiap heuristic berjalan lancar dan menghasilkan solusi yang dapat divisualisasikan melalui playback GUI.



Gambar 5: Bukti GUI A* H1 pada testcase_astar.txt



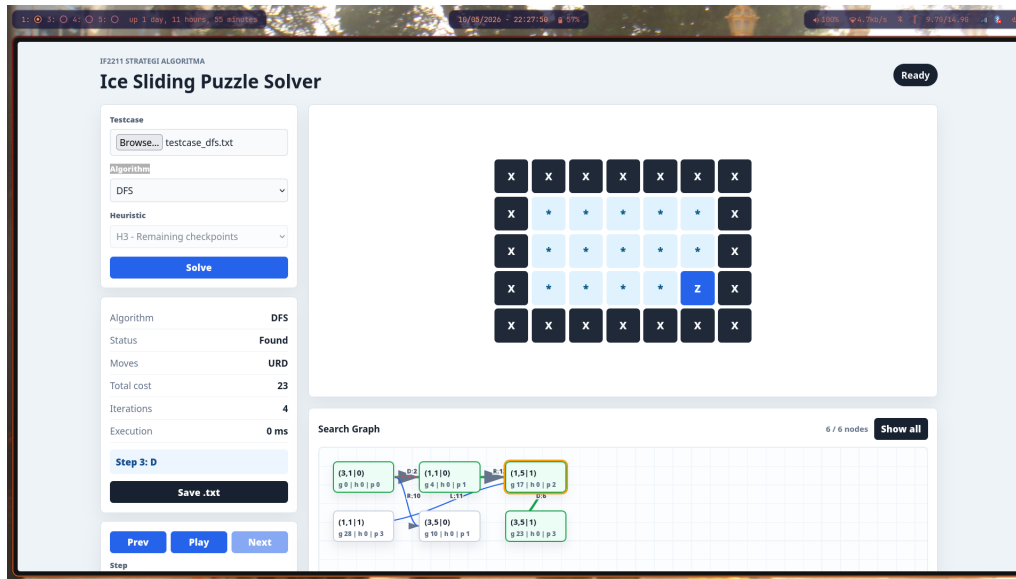
Gambar 6: Bukti GUI A* H2 pada testcase_astar.txt



Gambar 7: Bukti GUI A* H3 pada testcase_astar.txt

7.6 Kasus Uji 4: DFS pada testcase_dfs.txt

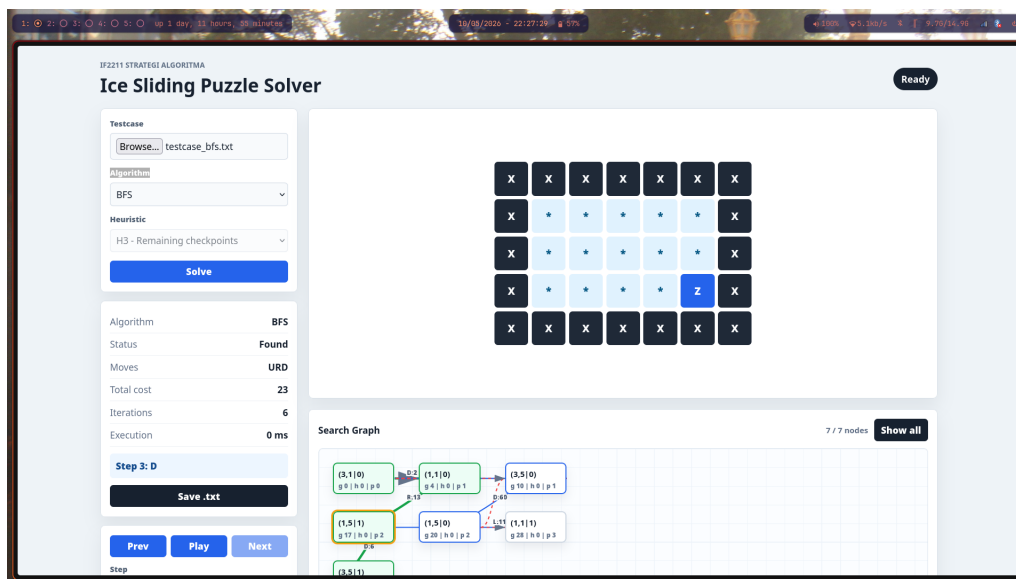
- File testcase: test/testcase_dfs.txt.
- Algoritma: DFS.
- Heuristic: tidak digunakan.
- Kompleksitas kasus: papan non-persegi 5×7 dengan checkpoint 0; digunakan untuk memastikan program tidak hanya berjalan pada papan persegi.
- Ekspektasi: GUI menampilkan solusi valid. Solusi DFS tidak harus optimal terhadap total cost maupun jumlah slide.



Gambar 8: Bukti GUI DFS pada testcase_dfs.txt

7.7 Kasus Uji 5: BFS pada testcase_bfs.txt

- File testcase: test/testcase_bfs.txt.
- Algoritma: BFS.
- Heuristic: tidak digunakan.
- Kompleksitas kasus: papan non-persegi dengan checkpoint, sehingga BFS diuji pada ruang status yang tidak hanya berisi start dan goal.
- Ekspektasi: GUI menampilkan solusi valid dengan jumlah slide minimum pada model pencarian BFS.



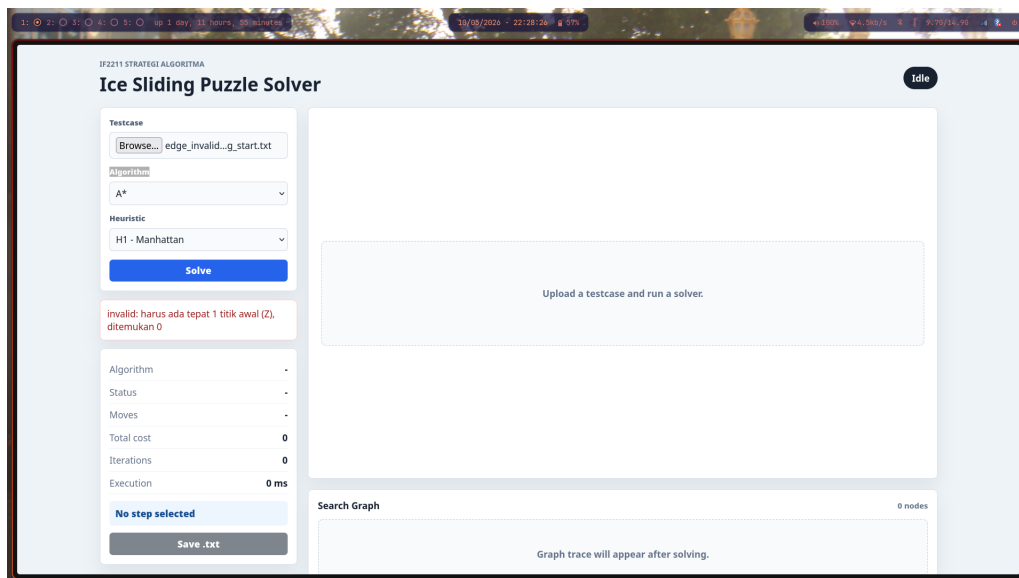
Gambar 9: Bukti GUI BFS pada testcase_bfs.txt

7.8 Pengujian Edge Case dan Error Handling

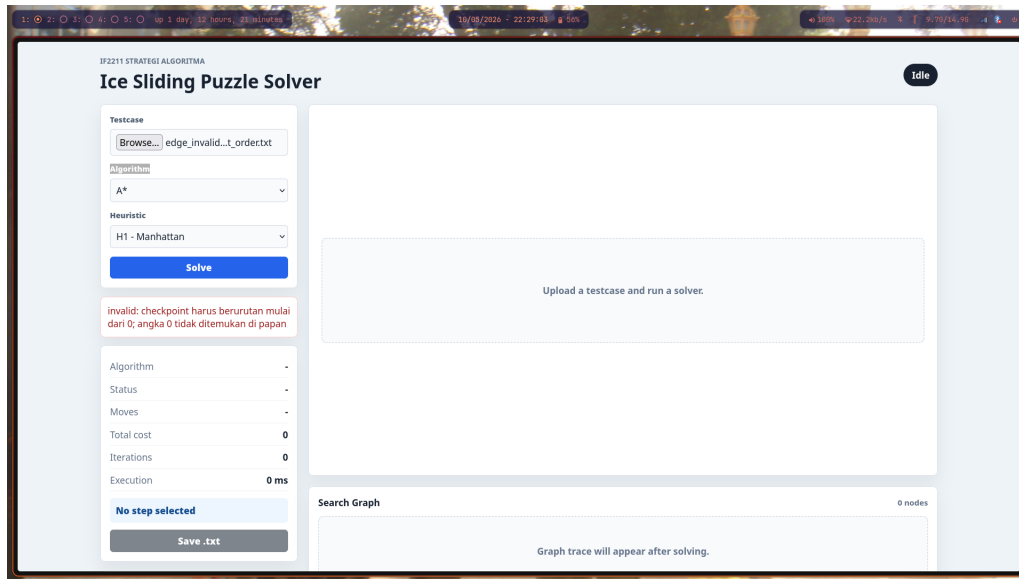
Selain testcase yang menghasilkan solusi, dilakukan juga pengujian terhadap edge case dan input tidak valid. Bukti untuk bagian ini juga dapat diambil dari GUI, yaitu dengan mengunggah file testcase dan memastikan pesan error atau status tidak ditemukan tampil dengan benar.

No	File	Ekspektasi
1	test/edge_no_solution.txt	Input valid, tetapi solusi tidak ditemukan karena area start dan goal terpisah oleh dinding.
2	Missing start	Parser menolak input <code>edge_invalid_missing_start.txt</code> karena tidak ada tepat satu Z.
3	Checkpoint order	Parser menolak input <code>edge_invalid_checkpoint_order.txt</code> karena checkpoint tidak berurutan mulai dari 0.
4	Cost count	Parser menolak input <code>edge_invalid_cost_count.txt</code> karena jumlah nilai cost tidak sesuai ukuran papan.

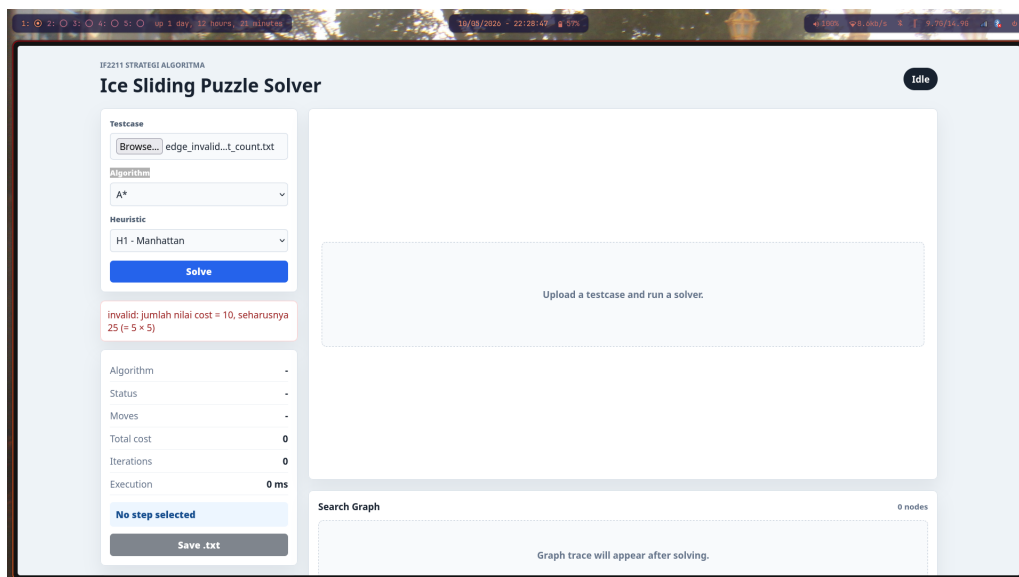
Tabel 5: Edge case dan error handling



Gambar 10: Bukti GUI error handling untuk input tanpa start



Gambar 11: Bukti GUI error handling untuk urutan checkpoint tidak valid



Gambar 12: Bukti GUI error handling untuk jumlah cost tidak valid

7.9 Ringkasan Hasil Pengujian

No	Kasus	Algoritma	Heuristic	Status GUI	Bukti
1	UCS testcase	UCS	-	Lulus	Gambar UCS
2	GBFS testcase	GBFS	H1, H2, H3	Lulus	Gambar GBFS H1-H3
3	A* testcase	A*	H1, H2, H3	Lulus	Gambar A* H1-H3
4	DFS testcase	DFS	-	Lulus	Gambar DFS
5	BFS testcase	BFS	-	Lulus	Gambar BFS
6	Edge case dan error handling	-	-	Lulus	Gambar error handling

Tabel 6: Ringkasan hasil pengujian GUI

8 Penutup

8.1 Kesimpulan

Berdasarkan rancangan dan implementasi yang telah dibuat, persoalan Ice Sliding Puzzle dapat dimodelkan sebagai pencarian lintasan pada graf berbobot. State permainan tidak hanya berisi posisi pemain, tetapi juga progres checkpoint. Dengan representasi tersebut, aturan checkpoint berurutan dapat ditangani secara alami oleh algoritma pencarian.

UCS digunakan sebagai algoritma pencarian berbasis biaya aktual dan menjadi acuan untuk solusi optimal jika seluruh biaya sisi tidak negatif. GBFS menggunakan heuristic sebagai prioritas sehingga dapat lebih cepat mengarah ke target, tetapi tidak menjamin optimalitas. A* menggabungkan biaya aktual dan estimasi heuristic sehingga menjadi kompromi antara optimalitas dan efisiensi pencarian. BFS dan DFS ditambahkan sebagai algoritma bonus. BFS optimal terhadap jumlah slide, sedangkan DFS berfungsi sebagai pembandingan strategi pencarian yang tidak menjamin optimalitas.

Selain CLI, program juga menyediakan GUI web. GUI membantu pengguna mengunggah testcase, memilih algoritma, melihat hasil solusi secara step-by-step, mengamati trace pencarian dalam bentuk graf, dan menyimpan ringkasan solusi ke berkas `.txt`.

8.2 Saran Pengembangan

Beberapa pengembangan yang dapat dilakukan selanjutnya adalah sebagai berikut.

1. Menambahkan lebih banyak heuristic dan menganalisis admissibility serta konsistensinya.
2. Mengoptimalkan visualisasi search graph agar tetap nyaman digunakan pada testcase besar.
3. Menambahkan pengujian otomatis untuk parser, aturan slide, dan solver.
4. Menambahkan build executable lintas sistem operasi agar folder `bin/` dapat langsung berisi executable untuk Windows dan Linux.

8.3 Lampiran

Tautan kode sumber program:

<https://github.com/raulinns/Tucil3-13524044>

Rangkuman keterselesaian program adalah sebagai berikut.

No	Poin	Ya	Tidak
1	Program dapat dikompilasi tanpa kesalahan	✓	
2	Program dapat dijalankan melalui CLI	✓	
3	Program dapat membaca masukan berkas <code>.txt</code>	✓	
4	Program mengimplementasikan UCS	✓	
5	Program mengimplementasikan GBFS	✓	
6	Program mengimplementasikan A*	✓	
7	Program mengimplementasikan BFS dan DFS sebagai algoritma bonus	✓	
8	Program memiliki alternatif heuristic H2 dan H3	✓	
9	Program menampilkan urutan solusi, cost, iterasi, dan waktu eksekusi	✓	
10	Program memiliki GUI web	✓	
11	GUI menampilkan playback solusi	✓	
12	GUI menampilkan trace graf pencarian	✓	
13	Program menyimpan solusi ke berkas <code>.txt</code>	✓	

Tabel 7: Ringkasan keterselesaian program

8.4 Kata Penutup

Tugas kecil ini memberikan pengalaman dalam memodelkan permainan sebagai persoalan graf dan membandingkan karakteristik algoritma pencarian berbeda. Implementasi juga memperlihatkan bahwa desain representasi state sangat mempengaruhi kesederhanaan solver, terutama ketika terdapat aturan tambahan seperti checkpoint berurutan.

8.5 Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 10 Mei 2026

A handwritten signature in black ink, consisting of a large, sweeping initial 'N' followed by a series of connected loops and a final horizontal stroke.

Narendra Dharma Wistara M.
13524044